HotelPro - Système de Gestion Hôtelière Multi-Hôtel Système professionnel de gestion hôtelière avec pré-réservations en ligne, QR codes, impression réseau automatisée et gestion multi-hôtel. ✨ Fonctionnalités Principales Multi-Hôtel · ✅ Gestion de plusieurs hôtels depuis une seule application · ✅ Isolation complète des données entre hôtels · ✅ Interface adaptée selon le rôle (Super Admin, Hotel Admin, Receptionist) Système de Réservations · ✅ Formulaires publics accessibles via QR code · ✅ Pré-réservations avec validation par l'hôtel · ✅ Gestion des groupes avec accompagnants · ✅ Documents d'identité enrichis (lieu et date de délivrance) · ✅ Signatures numériques 🖨 Impression Réseau Révolutionnaire · ✅ Test de connexion RÉEL aux imprimantes réseau · ✅ Envoi d'impressions RÉEL via ESC/POS · ✅ Scan automatique du réseau local · ✅ Support multi-fabricants (HP, Canon, Epson, Brother, Samsung, etc.) · ✅ Gestion avancée des erreurs avec retry automatique · ✅ Logs détaillés de chaque impression Fiches de Police · ✅ Génération automatique en PDF · ✅ Preview avant impression · ✅ Impression batch (plusieurs à la fois) · ✅ Envoi direct à l'imprimante réseau Sécurité · ✅ Authentification Laravel Breeze · ✅ Gestion des rôles et permissions (Spatie) · ✅ Protection CSRF · ✅ Rate limiting sur formulaires publics · ✅ Validation stricte des données 📊 Administration · ✅ Dashboard statistiques par hôtel · ✅ Gestion des utilisateurs · ✅ Journal d'activité · ✅ Rapports détaillés · ✅ Module de gestion des données (Reset, Export, Import, Purge) ⭐ Installation Prérequis · PHP 8.2 ou supérieur · Composer · Node.js & NPM · MySQL 8.0+ ou PostgreSQL · Redis (recommandé pour le cache) Étapes d'Installation 1. Cloner le dépôt git clone https://github.com/votre-repo/hotelpro.git cd hotelpro 2. Installer les dépendances PHP composer install 3. Installer les dépendances JavaScript npm install 4. Configuration de l'environnement cp .env.example .env php artisan key:generate 5. Configurer la base de données Éditer .env : DB_CONNECTION=mysql DB_HOST=127.0.0.1 DB_PORT=3306 DB_DATABASE=hotelpro DB_USERNAME=root DB_PASSWORD= # Cache Redis (recommandé) CACHE_DRIVER=redis SESSION_DRIVER=redis QUEUE_CONNECTION=redis # Mail MAIL_MAILER=smtp MAIL_HOST=smtp.mailtrap.io MAIL_PORT=2525 MAIL_USERNAME=null MAIL_PASSWORD=null 6. Exécuter les migrations php artisan migrate --seed 7. Créer un lien symbolique pour le storage php artisan storage:link 8. Compiler les assets npm run build # ou en mode développement npm run dev 9. Lancer le serveur php artisan serve L'application sera accessible sur http://127.0.0.1:8000 Comptes par Défaut Après le seeding : Rôle Email Mot de passe Super Admin admin@hotelpro.com password Hotel Admin hotel@example.com password Receptionist reception@example.com password Structure du Projet hotelpro/ ├── app/ │ ├── Http/Controllers/ │ │ ├── SuperAdmin/ # Contrôleurs Super Admin │ │ ├── HotelAdmin/ # Contrôleurs Hotel Admin │ │ ├── Reception/ # Contrôleurs Réception │ │ └── PrinterController # Gestion des imprimantes ⭐ │ ├── Models/ # Modèles Eloquent │ ├── Services/ # Services métier │ └── Jobs/ # Jobs asynchrones ├── database/ │ └── migrations/ # 35+ migrations ├── resources/ │ ├── views/ # Vues Blade │ └── js/ # JavaScript ├── routes/ │ └── web.php # Routes organisées par rôle ├── docs/ # 📚 Documentation complète │ ├── guides/ # Guides utilisateur │ ├── corrections/ # Historique corrections │ └── rapports/ # Rapports techniques └── tests/ # Tests Configuration des Imprimantes Scan Automatique du Réseau 1. Accéder à Impression > Gestion des imprimantes 2. Cliquer sur

Scanner le réseau 3. L'application détectera automatiquement les imprimantes disponibles 4. Cliquer sur Ajouter pour enregistrer une imprimante détectée Ajout Manuel 1. Nom : Nom de l'imprimante (ex: "Imprimante Reception") 2. Adresse IP : Adresse IP de l'imprimante (ex: 192.168.1.100) 3. Port : Port réseau (défaut: 9100 pour ESC/POS) 4. Type : thermique, laser, jet d'encre, multifonction 5. Module : reception, administration, cuisine Test de Connexion Cliquer sur le bouton Test connexion pour vérifier que l'imprimante est accessible. Impression de Test Cliquer sur Test impression pour envoyer un document de test à l'imprimante. 📊 Module de Gestion des Données (Super Admin) Accès Super Admin > Gestion des Données > Données Hôtels Fonctionnalités 1. 📥 Export · Télécharge toutes les données d'un hôtel au format JSON · Inclut : réservations, utilisateurs, imprimantes, logs · Utile pour backup ou migration 2. 📤 Import · Importe des données depuis un fichier JSON · Deux modes : o Fusionner : Ajouter aux données existantes o Remplacer : Supprimer et remplacer (dangereux !) 3. 🗑️ Purge · Supprime les anciennes données (soft clean) · Configurable par période (30-365 jours) · Types : pré-réservations annulées, logs d'impression, logs d'activité 4. ⚠️ Reset Complet · Réinitialise TOUTES les données d'un hôtel · ATTENTION : Irréversible ! · Nécessite confirmation avec le nom de l'hôtel · Option : supprimer les utilisateurs (sauf super-admin) ⚡ Optimisation des Performances Caches Laravel # En production php artisan config:cache php artisan route:cache php artisan view:cache php artisan optimize Redis Activer Redis dans .env : CACHE_DRIVER=redis SESSION_DRIVER=redis Indexes Database Les indexes de performance sont automatiquement créés avec la migration : php artisan migrate Queues Pour les tâches asynchrones (génération PDF, envoi emails) : php artisan queue:work En production, utiliser Supervisor pour gérer les workers (voir docs/guides/OPTIMISATION_PERFORMANCES.md). 📚 Documentation Toute la documentation est disponible dans le dossier docs/ : · · Guides : docs/guides/ o Installation o Configuration imprimantes o Optimisation performances o Tests · · Corrections : docs/corrections/ o Historique des bugs corrigés · · 📊 Rapports : docs/rapports/ o Rapports d'expertise o Analyses techniques Tests Tests Manuels Les scripts de tests manuels sont dans tests/manual/ Tests Automatisés php artisan test 🛠️ Technologies Utilisées Backend · Laravel 12.0 - Framework PHP · Spatie Laravel Permission - Gestion des rôles · DomPDF - Génération de PDF · SimpleSoftwareIO/simple-qrcode - QR codes · mike42/escpos-php - Impression ESC/POS thermique · Predis - Client Redis Frontend · Blade - Moteur de templates · TailwindCSS - Framework CSS · Alpine.js - Interactivité JavaScript · Flowbite - Composants UI · Vite - Build tool Database · MySQL ou PostgreSQL · Redis - Cache et sessions Contribution Les contributions sont les bienvenues ! Veuillez : 1. Fork le projet 2. Créer une branche (git checkout -b feature/AmazingFeature) 3. Commit vos changements (git commit -m 'Add AmazingFeature') 4. Push vers la branche (git push origin feature/AmazingFeature) 5. Ouvrir une Pull Request License Ce projet est sous licence MIT. Voir le fichier LICENSE pour plus de détails. 💻 Auteurs · Équipe HotelPro - Développement initial Support Pour toute question ou problème : · Email : support@hotelpro.com · Documentation : docs/ · Issues : GitHub Issues Roadmap · Application mobile (React Native) · API REST publique · Module de facturation · Intégration avec Oracle Hospitality · Système de notification push ·

Dashboard analytics avancé · Multi-langue (i18n) ☆ Fonctionnalités Uniques Ce projet se démarque par : 1. Système d'impression réseau révolutionnaire avec test RÉEL et envoi RÉEL aux imprimantes 2. Module de gestion des données pour reset/export/import/purge 3. Architecture multi-hôtel avec isolation complète 4. QR codes dynamiques pour formulaires publics 5. Sécurité maximale avec permissions granulaires Développé avec ❤ pour l'industrie hôtelière Documentation • Installation • Support je veux donc ajouter services des étages à ce qui existe et également services techniques et buanderie à ce dernier selon l'analyse faite plus haut:

Parfait 👍

Là on est clairement sur **une évolution d'un produit déjà solide**, donc je vais raisonner **en intégration propre**, sans casser ce qui existe, **en respectant ton architecture actuelle HotelPro** (multi-hôtel, rôles, modules, services, impressions, logs, etc.).

Je vais te répondre en **4 blocs clairs** :

1. **Positionnement des nouveaux services dans HotelPro existant**
2. **Nouvelles entités à ajouter (sans casser l'existant)**
3. **Fonctionnement détaillé par service (Étages, Technique, Buanderie)**
4. **Architecture Laravel concrète (dossiers, modules, règles, événements)**

Tout sera **cohérent avec ce que tu as déjà**.

1. POSITIONNEMENT DANS HOTELPRO EXISTANT

1.1 Ce qui existe déjà (base saine)

Tu as déjà :

- ✅ Multi-hôtel avec isolation des données
- ✅ Rôles & permissions (Spatie)
- ✅ Réception complète (réservations, fiches police, QR, impression)
- ✅ Logs, journaux, dashboard
- ✅ Architecture services / jobs / controllers claire

👉 **On NE TOUCHE PAS à la réception**

👉 On **se branche dessus**

1.2 Principe d'intégration

Les nouveaux services :

- **Service des Étages**
- **Service Technique**
- **Buanderie**

seront :

- des **modules indépendants**
- branchés via :
    - événements Laravel
    - états
    - règles paramétrables
- **désactivables par hôtel**

1.3 Activation par hôtel (clé du multi-hôtel)

Ajout dans la config hôtel :

```
hotel_settings = {
  "modules": {
    "housekeeping": true,
    "maintenance": true,
    "laundry": false
  }
}
```

☞ Un hôtel peut :

- ne pas avoir de buanderie
- externaliser la maintenance
- tout gérer manuellement

2. NOUVELLES ENTITÉS À AJOUTER (COMPATIBLES)

2.1 Chambres & espaces (extension)

Tu as déjà implicitement des chambres via la réception.
On **formalise proprement** :

`spaces`

```
id
hotel_id
type (room, corridor, office, kitchen, bar)
floor
code
active
```

`rooms`

```
space_id (FK)

category

capacity

base_price
```

☞ La réception continue d'utiliser les chambres **sans rien changer**.

2.2 États indépendants (fondamental)

`room_states`

```
id

room_id

occupation_state   (free, occupied, released)

cleaning_state     (none, pending, in_progress, done)

technical_state    (normal, issue, maintenance, out_of_service)

updated_at
```

☞ **La réception ne gère que `occupation_state`**

☞ Les autres services gèrent leurs états **sans collision**

2.3 Service des Étages

`housekeeping_tasks`

```
id

hotel_id

room_id

type (cleaning, inspection)

status (pending, in_progress, done)

assigned_to

created_at
```

housekeeping_products_usage

task_id

product_id

quantity

2.4 Buanderie

laundry_items

id

hotel_id

type (sheet, towel, client_clothes)

owner (hotel/client)

laundry_movements

id

laundry_item_id

source (reception, room, housekeeping)

destination (laundry)

status (received, washing, ready)

2.5 Service Technique

maintenance_issues

id

hotel_id

space_id

category (plumbing, AC, electricity)

description

priority

status (reported, in_progress, resolved)

maintenance_interventions

id

`issue_id`

`technician_id`

`start_time`

`end_time`

3. FONCTIONNEMENT PAR SERVICE (INTÉGRÉ À LA RÉCEPTION)

3.1 Réception → Service des Étages

Cas : Libération de chambre

1. La réception libère la chambre (déjà existant)
2. **Événement Laravel émis** :

```
RoomReleased::dispatch($room);
```

1. Règle paramétrable :
   - créer tâche de nettoyage ? oui / non
   - notifier housekeeping ? oui / non
1. Si activé :
   - `cleaning_state = pending`
   - création `housekeeping_task`

☞ Désactiver housekeeping = aucun impact réception

3.2 Service des Étages → Buanderie

Cas : Nettoyage terminé

- La femme de chambre :
  - marque linge récupéré
  - enregistre produits utilisés
- Si buanderie activée :
  - mouvement linge créé
  - notification buanderie

☞ Sinon : historique simple

3.3 Service des Étages → Service Technique

Cas : panne détectée

- Création `maintenance_issue`

- Catégorie + priorité
- Règle :
    - mettre chambre HS ? oui / non
    - notifier maintenance ? oui / non

☞ La réception voit **l'état**, pas le détail technique

3.4 Service Technique → Réception

Cas : panne résolue

- `technical_state = normal`
- Règle :
    - rendre chambre disponible ? oui / non
- Notification réception (optionnelle)

3.5 Buanderie → Service des Étages

- Linge prêt
- Notification housekeeping
- Clôture du cycle nettoyage

4. ARCHITECTURE LARAVEL (PROPRE & ALIGNÉE)

4.1 Dossiers à ajouter

```
app/
├── Modules/
│   ├── Housekeeping/
│   │   ├── Controllers/
│   │   ├── Models/
│   │   ├── Services/
│   │   └── Events/
│   ├── Maintenance/
│   │   ├── Controllers/
│   │   ├── Models/
│   │   ├── Services/
│   │   └── Events/
│   └── Laundry/
│       ├── Controllers/
│       ├── Models/
```

```
│        ├── Services/
│        └── Events/
```

4.2 Core partagé (important)

```
app/Core/
├── RulesEngine/
├── StateManager/
├── EventBus/
├── SettingsResolver/
```

👉 Réutilisable pour TOUS les modules futurs

4.3 Événements clés

- `RoomOccupied`
- `RoomReleased`
- `CleaningStarted`
- `CleaningCompleted`
- `IssueReported`
- `IssueResolved`
- `LaundryReceived`
- `LaundryReady`

4.4 Permissions Spatie

Exemples :

- `housekeeping.view`
- `housekeeping.update`
- `maintenance.report`
- `maintenance.resolve`
- `laundry.manage`

5. CE QUE TU OBTIENS AU FINAL

✅ Intégration **sans casser la réception**
✅ Modules activables par hôtel
✅ États indépendants (propre & robuste)
✅ Traçabilité totale
✅ Compatible QR, impressions, logs
✅ Prêt pour mobile & API
✅ Produit **vendable**